

ENTERPRISE FIELD OPERATIONS PLATFORM

Detailed Project Report - Version 2.0

A Comprehensive Blueprint for Design, Development & Deployment

Project Name	Enterprise Field Operations Platform
Report Version	2.0 - Full Detail
Prepared For	Paramesh Anna - Project Sponsor
Prepared By	Technical Architecture Team
Classification	Confidential - Demo Copy
Date	5 May 2026
Total Modules	8 Functional Modules
Est. Duration	26 Weeks ~6.5 Months
Est. Team Size	6-8 Members (Full Stack + QA + DevOps)

Table of Contents

Table of Contents	2
1. Executive Summary.....	4
2. Project Objectives & Success Criteria	5
2.1 Primary Objectives	5
2.2 Success Criteria	5
3. User Roles, Personas & Access Control	6
3.1 Role Definitions	6
3.2 RBAC Permission Matrix	6
4. Detailed Module Specifications	7
4.1 Employee Task Manager	7
Screens & Navigation	7
Business Rules	7
Task Status Workflow	7
Key Data Fields	7
4.2 Expenses Manager	8
Screens & Navigation	8
Expense Categories	8
Approval Workflow	9
4.3 Service Manager	9
Service Call Lifecycle	9
SLA Tiers	10
Auto-Email Template: Service Completion	10
4.4 Documents Management	10
Folder Structure & Access	10
Document Lifecycle	10
4.5 Work Reporting	10
Daily Report Fields	11
Reporting Dashboard Visuals	11
4.6 Daily Task Manager - Advanced View	12
Efficiency Calculator - Detailed Logic	12
4.7 Chatbot & Automated Notification System	12
Chatbot - Supported Query Types	12
Automated Notification Triggers	13
4.8 Electronic Sign Approvals & Rejection Workflow	13
Signature Methods	13
Audit Trail Fields (per approval event)	13
5. Modular Home Page Design	15
5.1 Tile Specifications	15
5.2 Navigation Structure	15

6. Technology Stack - Detailed.....	16
7. System Architecture.....	18
7.1 Architecture Overview	18
7.2 Component Description	18
7.3 FastAPI Backend Project Structure	19
7.4 Key FastAPI Patterns Used	19
8. Database Design	20
8.1 Entity Relationship Summary.....	20
8.2 Core Table Schemas	20
users table	20
tasks table	21
9. API Specification.....	22
9.1 Authentication	22
9.2 Tasks API.....	22
9.3 Service Calls API	22
10. Security Architecture	24
11. Project Timeline & Sprint Plan	25
11.1 Phase Overview	25
11.2 Team Composition	25
12. Estimated Infrastructure Cost	26
13. Risk Register	27
14. Testing Strategy.....	28
15. Recommended Next Steps.....	29

1. Executive Summary

The Enterprise Field Operations Platform is a modular, cloud-native web application designed to unify and digitise the core operational workflows of field-service and operations-driven organisations. It replaces disconnected spreadsheets, paper-based approvals, and fragmented communication channels with a single, role-aware digital workspace.

The platform delivers eight tightly integrated functional modules covering employee task management, expense lifecycle tracking, service call operations, document management, work reporting, AI-powered chatbot assistance, automated notifications, and electronic approval workflows, all accessible from a personalised, modular home page.

Business Problem: Most field-service companies manage tasks in WhatsApp groups, expenses in Excel, service calls via phone calls, and approvals on paper. This causes delays, lost records, zero visibility, and no performance data.

Our Solution: One platform. Every workflow digitised. Every action tracked. Every approval audited. Available on web and mobile browser, deployable on Microsoft Azure in under a day.

Modules	Timeline	Team Size	Tech Platform
8 Modules	26 Weeks	6-8 Members	Azure Cloud

2. Project Objectives & Success Criteria

2.1 Primary Objectives

- **Eliminate paper-based task assignments and approvals across all departments**
- Provide real-time visibility into employee performance using graphical efficiency dashboards
- Automate the full expense lifecycle from submission through multi-level approval to reconciliation
- Centralise service call management with coordinator-driven assignment and closure workflows
- Create a secure, versioned document repository with electronic signature capability
- Automate customer-facing emails on service completion and SMS alerts to field technicians
- Embed an AI chatbot for guided navigation, FAQs, and live data queries
- Enforce role-based access with immutable audit trails for every approval and rejection

2.2 Success Criteria

Success Metric	Target	Measurement
Task assignment to acknowledgement time	< 5 minutes (vs. current avg 2 hrs via WhatsApp)	Platform logs
Expense approval turnaround	< 24 hours for single-level approval	Approval timestamps
Service call SLA breach rate	< 5% of calls breach SLA	SLA dashboard
Document retrieval time	< 30 seconds via search	Search analytics
User adoption rate	> 80% of employees active within 4 weeks of go-live	Login analytics
System uptime	99.5% monthly uptime	Azure Monitor
Report generation time	Daily reports generated in < 10 seconds	Performance logs

3. User Roles, Personas & Access Control

3.1 Role Definitions

Role	Typical User	Platform Access & Permissions
Super Admin	IT Manager / Owner	Full system access. User management, system config, all module data, reports, audit logs. Cannot be restricted.
Admin	Department Head / HR Manager	All module access. Approves/rejects tasks, expenses, documents. Manages employee profiles. Views all reports.
Supervisor	Team Lead / Field Supervisor	Views and manages team tasks. Approves task completions. Reviews work reports. Sees team efficiency scores.
Service Coordinator	Operations Coordinator	Creates and assigns service calls. Confirms closure. Views all open/active service calls. Sends client notifications.
Finance Officer	Accounts Executive	Full access to Expenses module. Approves/rejects expense claims. Views payables/receivables dashboard. Exports reports.
Employee	Field Technician / Office Staff	Creates own tasks, submits expenses, uploads documents, submits daily work reports. Views own performance only.
Viewer / Auditor	External Auditor / Compliance	Read-only access to reports and documents. Cannot create or modify any records. Audit log access.

3.2 RBAC Permission Matrix

Module	S.Admin	Admin	Supvsr	Coord.	Finance	Empl.	Viewer
Task Manager	Full	Full	Team	--	--	Own	Read
Expenses Manager	Full	Full	View	--	Full	Own	Read
Service Manager	Full	Full	View	Full	--	View	Read
Documents Mgmt	Full	Full	Team	--	--	Own	Read
Work Reporting	Full	Full	Team	--	--	Own	Read
Approvals / E-Sign	Full	Full	Tasks	--	Exps	--	Read
Chatbot	Full	Full	Full	Full	Full	Full	Full
User Management	Full	Full	--	--	--	--	--
Audit Logs	Full	Full	--	--	--	--	Read
System Config	Full	--	--	--	--	--	--

Full=complete CRUD | Own=own records only | Team=assigned team only | Read=view only | --=no access

4. Detailed Module Specifications

Each module below is described with its purpose, screen inventory, detailed feature list, business rules, workflow, data fields, and notification triggers.

4.1 Employee Task Manager

Purpose: Centralise daily work assignments, track progress in real-time, and measure employee efficiency through graphical dashboards and delay-aware scoring.

Screens & Navigation

Screen	Description
Task Dashboard	Overview cards: Total Tasks, Completed, In Progress, Overdue. Efficiency score badge. Quick filters.
My Tasks List	Paginated list view with status badges, due dates, priority flags. Sortable by date, priority, status.
Create / Edit Task	Form: Title, Description, Assignee (multi-select), Department, Priority, Due Date, Attachments, Tags.
Task Detail View	Full task info, comment thread, status history timeline, attached files, approval status.
Daily Task View	Day-at-a-glance timeline. Animated completion ring. Drag-and-drop reorder. Delay counter.
Graphical Reports	Bar chart: tasks by status. Line chart: efficiency trend (7/30 days). Pie: tasks by category.
Approval Queue	Admin/Supervisor queue of tasks awaiting review. Approve/Reject with e-signature.
Team Overview	Supervisor-only: all team members' task counts, efficiency scores, overdue alerts in a grid.

Business Rules

- A task must have a title, at least one assignee, a due date, and a priority level before it can be saved
- Only the task creator or a Supervisor/Admin can reassign or edit a task after creation
- Efficiency Score = (On-Time Completions / Total Assigned) x 100, weighted by Priority (Critical=3x, High=2x, Normal=1x)
- A task overdue by more than 24 hours triggers an automatic escalation email to the Supervisor
- Tasks cannot be marked Complete without uploading at least one completion note or attachment
- Completed tasks require Supervisor or Admin e-sign approval before the record is finalised

Task Status Workflow

Flow: New > Assigned > In Progress > Pending Review > Approved (Closed) | Any state > Rejected (returns to Assigned)

Key Data Fields

Field	Type	Validation / Notes
task_id	UUID (PK)	Auto-generated
title	VARCHAR(200)	Required, min 5 chars
description	TEXT	Optional, max 2000 chars
assigned_to	FK users[]	Required, supports multi-assignee (junction table)
assigned_by	FK users	Auto-set to logged-in user
priority	ENUM	Critical / High / Normal / Low
status	ENUM	New / Assigned / In Progress / Pending Review / Approved / Rejected
due_date	DATETIME	Required, must be future date
completed_at	DATETIME	Set when status changes to Pending Review
delay_minutes	INT	Computed: completed_at minus due_date (negative = early)
efficiency_score	DECIMAL(5,2)	Computed on approval, stored for reporting
attachments	BLOB URLs[]	Azure Blob Storage URLs, max 10 files, 20MB each
tags	VARCHAR[]	Optional, free-form, used for filtering
approved_by	FK users	Supervisor or Admin who e-signed
approved_at	DATETIME	Timestamp of approval/rejection

4.2 Expenses Manager

Purpose: Track all business expenses through a structured digital workflow from employee submission through supervisor and finance approval, with live receivables and payables tracking.

Screens & Navigation

Screen	Description
Expenses Dashboard	Summary cards: Total Submitted, Approved, Pending, Rejected. Net Balance (Receivable minus Payable).
Submit Expense	Form: Type (Payable/Receivable), Category, Amount, Currency, Date, Vendor, Receipt Upload, Notes.
My Expenses List	Status-filtered list. Click to expand details and view approval history.
Finance Dashboard	Finance Officer view: all pending claims, payables vs. receivables chart, ageing analysis.
Approval Queue	Supervisor then Finance two-level queue. Each level sees only claims at their stage.
Reports & Export	Monthly summary, category breakdown bar chart. Export to Excel and PDF.
Expense Detail	Full claim view: all fields, receipt preview, approval chain with timestamps and signatures.

Expense Categories

Payable Categories	Receivable Categories	Limits & Rules
Travel & Transport	Client Reimbursements	Single claim max: Rs.50,000
Accommodation	Project Billings	Receipts mandatory above Rs.500
Equipment Purchase	Vendor Refunds	Finance approval above Rs.10,000
Office Supplies	Grant Receipts	Two-level approval above Rs.25,000
Client Entertainment	Deposit Returns	Monthly limit per employee configurable
Training & Certification	Insurance Claims	Automatic rejection if duplicate detected

Approval Workflow

Single-level (up to Rs.10,000): Employee > Supervisor Approval > Finance Notification > Processed

Two-level (above Rs.10,000): Employee > Supervisor Approval > Finance Officer Approval > Processed

Rejection at any level: Mandatory rejection reason stored. Employee notified by email. Claim returned with reason for resubmission.

4.3 Service Manager

Purpose: Manage the complete field service call lifecycle from client complaint through technician assignment, field resolution, and coordinator-confirmed closure with automated client communications.

Service Call Lifecycle

Stage	Actor	Actions & System Events
1. New	Any user / Client portal	Service call created with client info, issue description, priority, SLA deadline. Auto-assigned a ticket number.
2. Pending Assignment	Service Coordinator	Coordinator reviews call, checks technician availability, assigns to best-fit technician. SMS sent to technician.
3. Assigned	Technician	Technician acknowledges assignment via app. Status updates to Assigned. Client notified by email.
4. In Progress	Technician	Technician marks call In Progress on arrival. GPS location optionally logged. Timer starts.
5. Resolved	Technician	Technician marks resolved, adds resolution notes, uploads photos/documents as proof of work.
6. Closed	Service Coordinator	Coordinator reviews resolution, confirms closure. Auto-email sent to client with summary. SLA compliance logged.
7. Escalated	Supervisor	If SLA breach imminent (>80% of deadline elapsed), auto-escalation email to Supervisor. Call flagged red.

SLA Tiers

Priority	Response SLA	Resolution SLA	Escalation Trigger
Critical	30 minutes	4 hours	At 80% of deadline (3h 12m)
High	2 hours	8 hours	At 80% of deadline (6h 24m)
Medium	4 hours	24 hours	At 80% of deadline (19h 12m)
Low	8 hours	72 hours	At 80% of deadline (57h 36m)

Auto-Email Template: Service Completion

Subject: Your Service Request #[TICKET_ID] has been Completed

Dear [CLIENT_NAME], Your service request ([ISSUE_TYPE]) logged on [DATE] has been successfully resolved. Technician: [TECH_NAME] | Resolution: [RESOLUTION_NOTES] | Closed On: [CLOSE_DATE]. Please rate your experience: [FEEDBACK_LINK]. For further support, contact [SUPPORT_EMAIL].

4.4 Documents Management

Purpose: Provide a secure, structured, version-controlled document repository with role-based access, electronic signing, and full audit trail.

Folder Structure & Access

- Root folders defined by Admin: HR Documents, Finance, Service Records, Client Files, Policies, Projects
- Sub-folders creatable by Admin and Supervisor; employees upload into assigned folders only
- Each folder has an Access List: roles and specific users who can read, write, or manage
- Documents inherit folder permissions; individual override possible per document

Document Lifecycle

Stage	Description
Upload	User uploads file (PDF, DOCX, XLSX, PNG, JPG). Max 50MB. Version 1.0 auto-created.
Review	Admin or Supervisor reviews document. Can annotate with comments.
E-Sign Request	Uploader or Admin sends e-sign request to specific users. Deadline optional.
Signing	Recipients open document, draw or type signature on canvas, confirm. Timestamp recorded.
Approved / Rejected	Signed document locked (read-only). Rejected document returned with reason.
Archival	Documents older than configured period auto-archived. Still searchable but not editable.
Audit Log	Every view, download, edit, sign event logged: who, what, when, from which IP.

4.5 Work Reporting

Purpose: Enable employees to submit structured daily work logs and provide management with aggregated performance visibility through charts and exportable reports.

Daily Report Fields

Field	Type	Details
Report Date	DATE	Auto-filled (today). Employee cannot backdate more than 2 days.
Employee	FK users	Auto-filled from session.
Tasks Worked On	FK tasks[]	Multi-select from employee's active tasks.
Hours Logged	DECIMAL(4,1)	Per task, sum must be 12 hours or less.
Activities Summary	TEXT	Free-text description of work done, max 1000 chars.
Blockers / Issues	TEXT	Optional, max 500 chars. If filled, Supervisor notified.
Tomorrow's Plan	TEXT	Optional forward-looking field for planning.
Mood / Workload	ENUM	Light / Normal / Heavy / Overloaded (for HR analytics).
Attachments	BLOB URLs[]	Optional: screenshots, deliverables, up to 5 files.

Reporting Dashboard Visuals

- Daily Submission Rate: percentage of employees who submitted their report today (gauge chart)
- Team Activity Heatmap: hours logged per employee per day (calendar grid)
- Blocker Frequency Chart: how often blockers are reported per department
- Top Performers: ranked by tasks completed plus hours logged (configurable period)
- Export: full team report as Excel (one row per employee-day) or PDF summary

4.6 Daily Task Manager - Advanced View

Purpose: A focused, day-level task interface with animated visual feedback, real-time delay tracking, and efficiency analytics at both individual and team level.

Efficiency Calculator - Detailed Logic

Component	Formula / Logic
Base Score	$(\text{Completed On-Time} / \text{Total Assigned}) \times 100$
Priority Weight	Critical tasks multiply contribution by 3x; High by 2x; Normal by 1x; Low by 0.5x
Delay Penalty	For each delayed task: deduct $(\text{Delay Hours} / \text{SLA Hours}) \times 10$ points from base score
Early Bonus	Each task completed more than 20% ahead of deadline adds 2 bonus points (max 10 bonus pts/day)
Final Score	$\text{Min}(100, \text{Base Score minus Delay Penalty plus Early Bonus})$, floored at 0
Rating Band	90-100: Excellent 75-89: Good 60-74: Average below 60: Needs Improvement
Trend	7-day rolling average displayed as sparkline on employee profile and home tile

4.7 Chatbot & Automated Notification System

Purpose: Provide instant AI-driven assistance within the platform and automate all outbound communications triggered by defined business events.

Chatbot - Supported Query Types

Query Category	Example Queries & Chatbot Response
Task Queries	'What are my pending tasks?' returns tasks with due dates. 'Show overdue tasks' returns filtered list with delay info.
Expense Queries	'What is my pending expense status?' shows claim status. 'How much have I claimed this month?' shows monthly total.
Service Call Queries	'Show open service calls in my area' returns filtered list. 'Status of ticket #1234' returns live status.
Document Queries	'Find the leave policy document' returns document link. 'Show documents pending my signature' returns pending list.
Reporting Queries	'Did I submit today's report?' gives Yes/No with prompt to submit. 'Show my efficiency this week' shows score and trend.
Guided Actions	'I want to submit an expense' opens expense form pre-filled with user details.
FAQ / Help	'How do I create a service call?' returns step-by-step guide. Pulls answers from Documents module.
Escalation	If chatbot cannot answer with above 75% confidence, it offers to escalate to a human supervisor via in-app alert.

Automated Notification Triggers

Trigger Event	Email	SMS	Recipients
Service call assigned to technician	Yes	Yes	Assigned technician
Service call closed by coordinator	Yes	No	Client + Finance Officer
SLA breach imminent (above 80% elapsed)	Yes	Yes	Supervisor + Coordinator
Expense claim submitted	Yes	No	Supervisor + Finance Officer
Expense approved or rejected	Yes	No	Submitting employee
Task overdue by 24 hours	Yes	No	Assignee + Supervisor
Task approval/rejection	Yes	No	Task creator
Document signature requested	Yes	No	Signing user
Daily report not submitted by 6 PM	Yes	No	Employee
Weekly efficiency score drops more than 15%	Yes	No	Employee + Supervisor
New user account created	Yes	No	New user

4.8 Electronic Sign Approvals & Rejection Workflow

Purpose: A cross-platform, legally-traceable approval engine supporting both drawn and typed electronic signatures with immutable audit trails for compliance.

Signature Methods

- **Draw Signature:** touchscreen or mouse freehand signature on HTML5 canvas stored as SVG and PNG
- **Type Signature:** typed full name rendered in a cursive font, stored with metadata (font, timestamp, user-agent)
- **OTP Verification:** optional 2nd-factor where user receives OTP via SMS to confirm identity before signing

Audit Trail Fields (per approval event)

Field	Type	Description
approval_id	UUID	Unique record per approval event
module	ENUM	task expense document service_call
ref_id	FK	ID of the parent record being approved
actor_id	FK users	Who performed the approval/rejection
action	ENUM	approved rejected escalated

signature_data	TEXT/URL	SVG string or PNG blob URL of drawn signature
signature_type	ENUM	drawn typed otp-verified
rejection_reason	TEXT	Mandatory if action = rejected, max 500 chars
ip_address	VARCHAR	Requester IP for security logging
user_agent	VARCHAR	Browser/device info
timestamp	DATETIME	Server-side UTC timestamp (cannot be client-manipulated)
hash	VARCHAR	SHA-256 hash of (ref_id + actor_id + action + timestamp) for tamper detection

5. Modular Home Page Design

The home page renders as a responsive, role-filtered tile grid. Each tile is a live widget that pulls real-time data on page load. Users can drag and drop tiles to customise their layout with preferences saved per user.

5.1 Tile Specifications

Module Tile	Tile Size	Live Data & Quick Actions
Task Manager	Wide (2x1)	Pending count, Due Today, Overdue alert, Efficiency badge. Quick-action: [+ New Task]
Daily Tasks	Wide (2x1)	Animated completion ring (today), 7-day efficiency sparkline, Delay counter.
Expenses	Standard (1x1)	Pending claims, Receivables total, Payables total. Quick-action: [Submit Expense]
Service Manager	Wide (2x1)	Open calls count, My Assigned calls, SLA breach alerts (red badge). Quick-action: [+ New Call]
Documents	Standard (1x1)	Recent uploads (last 3), Pending signatures count. Quick-action: [Upload]
Work Reports	Standard (1x1)	Today's report status (Submitted or Not Submitted). Quick-action: [Submit Report]
Notifications	Standard (1x1)	Unread count, last 3 notification previews. Quick-action: [View All]
Approvals	Wide (2x1)	(Admin/Supervisor only) Pending sign-offs by module. Quick-action: [Review Now]
Chatbot	Floating Widget	Persistent floating button on all screens. Opens slide-over chat panel.

5.2 Navigation Structure

- Top Navigation Bar: Logo, Global Search, Notifications Bell, User Avatar (Profile / Logout)
- Left Sidebar: Module links with unread/pending badge counts. Collapses to icon-only on smaller screens.
- Home Page: Default landing page. Role-filtered tile grid. Draggable and collapsible tiles.
- Breadcrumbs: Every inner page shows path. Back navigation always available.
- Responsive: Full-feature on desktop (1200px+), optimised on tablet (768px), core features on mobile.

6. Technology Stack - Detailed

Layer	Technology	Version / Plan	Justification
Frontend	React.js	v18+ with Vite	Component reuse, huge ecosystem, fast build times
UI Framework	Tailwind CSS + shadcn/ui	Latest	Rapid, consistent, accessible UI
State Management	Zustand / React Query	Latest	Lightweight global state + server caching
Charts	Recharts + Chart.js	Latest	Rich charts for dashboards and reports
E-Signature	react-signature-canvas	Latest	HTML5 canvas signature drawing
Backend	Python + FastAPI	v3.12 + FastAPI 0.110+	Async-native, auto OpenAPI docs, Pydantic validation
ORM / DB Driver	SQLAlchemy 2.0 + asyncpg	Latest	Async ORM for PostgreSQL, type-safe queries, Alembic migrations
Data Validation	Pydantic v2	Latest	Request/response models, auto validation, serialisation
Task Queue	Celery + Redis	Latest	Background jobs: report export, bulk notifications
API Server	Uvicorn + Gunicorn	Latest	ASGI server, multi-worker production deployment
Auth	Azure AD B2C	Standard tier	Enterprise identity, MFA, social login support
Primary DB	Azure SQL (PostgreSQL)	GP_Gen5_2	Relational, ACID, managed backups
File Storage	Azure Blob Storage	LRS + CDN	Document storage, cheap, globally fast
Cache	Azure Cache for Redis	Basic C1	Session, API response caching
Email	SendGrid	Essentials (40k/month)	Transactional email, delivery analytics
SMS	Twilio	Pay-as-you-go	SMS alerts, OTP delivery
Chatbot	Azure Bot Service + OpenAI GPT-4o	Standard	Conversational AI with company data grounding
Search	Azure Cognitive Search	Basic tier	Full-text document and task search
Monitoring	Azure App Insights + Monitor	Standard	Telemetry, alerts, performance profiling
CI/CD	GitHub Actions + Azure DevOps	Free/Basic	Automated pytest, Ruff linter, mypy type check, build Docker image, deploy to Azure
Hosting	Azure App Service	B2 then P1V3	Managed hosting, easy scaling, deployment slots

7. System Architecture

7.1 Architecture Overview

The platform follows a three-tier, microservices-leaning architecture deployed entirely on Microsoft Azure. The frontend is a React Single Page Application (SPA) served via Azure Static Web Apps or App Service. All API communication is over HTTPS with JWT authentication enforced at the API gateway level.

Architecture Pattern: Modular Monolith backend (single deployable Python FastAPI application) in Phase 1, with clear APIRouter-based module boundaries enabling extraction into microservices in Phase 2 if scale demands.

7.2 Component Description

Component	Details
React SPA (Client)	Served from Azure CDN. Communicates with API via REST over HTTPS. Auth tokens stored in memory (not localStorage). Refresh tokens in secure httpOnly cookies.
API Gateway / App Service	Python FastAPI on Azure App Service (Uvicorn + Gunicorn). JWT validation via python-jose middleware. Rate limiting via slowapi. Request logging to App Insights via opencensus-ext-azure. APIRouter per module for clean separation.
Module Routers (Controllers)	Separate APIRouter files per module (tasks.py, expenses.py, service_calls.py, etc.). Business logic in service layer. Data access via SQLAlchemy async repositories. Pydantic schemas enforce request/response shapes with auto validation.
Azure Functions (Python)	Triggered by: timer (daily digest), Service Bus queue (notifications), HTTP (report export). Python worker functions. Stateless, auto-scaled, pay-per-execution. Celery also used for heavy background jobs on the App Service.
Azure SQL Database	Primary data store. SQLAlchemy 2.0 async engine with asyncpg driver. Connection pooling built-in. Alembic for schema migrations (version-controlled). Read replica for reporting queries. Automated backups (7-day retention on Basic, 35-day on Standard+).
Azure Blob Storage	Document uploads. Pre-signed URLs for secure direct client upload (bypasses API server). Container-per-module structure. CDN-fronted for fast read.
Azure Service Bus	Decouples notification sending from API response. API publishes events (e.g., ServiceCallClosed). Azure Function consumes and dispatches email/SMS. Retry policy: 3 attempts with exponential backoff.
Azure AD B2C	Identity provider. Issues JWT access tokens (15-min expiry) and refresh tokens (7-day expiry). Supports MFA. RBAC roles embedded as custom claims in token.
Azure Bot Service	Hosts chatbot. Integrates with OpenAI GPT-4o via Azure OpenAI Service. Grounded with company documents from Azure Cognitive Search index. Fallback to human escalation.
Azure App Insights	Captures all API request telemetry, exceptions, custom events (e.g., ExpenseApproved). Enables end-to-end transaction tracing. Dashboard for live metrics.

7.3 FastAPI Backend Project Structure

The backend follows a clean layered architecture using FastAPI's APIRouter for modular organisation. Each functional module maps to a sub-package with its own router, schemas, service, and repository.

```
backend/  app/      main.py          # FastAPI app init, middleware, router registration
core/    config.py  # Settings via pydantic-settings (.env loading) security.py
# JWT decode, password hashing (passlib) dependencies.py # get db(), get current user()
FastAPI deps db/    base.py          # SQLAlchemy async engine + session factory
models/   # SQLAlchemy ORM models (one file per domain) modules/ auth/
router.py # APIRouter: /auth/login, /refresh, /logout, /me schemas.py #
Pydantic request/response models service.py # Business logic repository.py #
# Async DB queries tasks/ # Same structure: router, schemas, service,
repository expenses/ service calls/ documents/ reports/ approvals/
chatbot/ workers/ celery_app.py # Celery app config tasks/
# Background task functions (email, SMS, reports) tests/ # pytest test suite
mirroring modules/ structure alembic/ # DB migration scripts requirements.txt
Dockerfile
```

7.4 Key FastAPI Patterns Used

Pattern	Implementation
Dependency Injection	get_current_user() dependency injected into every protected route. get_db() provides async SQLAlchemy session. Role guards implemented as reusable dependencies.
Pydantic Schemas	Separate schemas per operation: TaskCreate, TaskUpdate, TaskResponse, TaskListResponse. Response models exclude sensitive fields automatically.
Async Throughout	All route handlers, service functions, and DB queries are async/await. asyncpg driver enables true async DB I/O. No blocking calls in the event loop.
Background Tasks	FastAPI BackgroundTasks for lightweight post-response work (e.g., log audit event). Celery for heavier jobs (PDF generation, bulk email dispatch).
Auto Documentation	FastAPI auto-generates OpenAPI 3.0 schema. All endpoints, request bodies, response models, and error codes documented at /docs with zero extra effort.
Exception Handling	Global exception handlers registered in main.py. HTTPException for API errors. Custom AppException classes for domain errors with structured JSON responses.
Middleware	CORS middleware, request ID injection, Azure App Insights telemetry middleware, rate limiting via slowapi all registered in main.py lifespan.

8. Database Design

8.1 Entity Relationship Summary

The database uses PostgreSQL (Azure SQL) with 18 primary tables organised into 6 domain schemas. Foreign keys enforce referential integrity. Soft deletes (deleted_at field) are used throughout and no hard deletes are performed.

Schema	Tables	Key Relationships
auth	users, roles, user_roles, sessions	users and roles are Many-to-Many via user_roles. sessions FK references users.
tasks	tasks, task_assignees, task_comments, task_attachments	tasks and users are Many-to-Many via task_assignees. comments and attachments FK reference tasks.
expenses	expenses, expense_attachments, expense_approvals	expenses FK references users. approvals FK references expenses and users.
service	service_calls, service_history, service_attachments	service_calls FK references users (assigned_to, coordinator). history logs every status change.
documents	documents, folders, doc_signatures, doc_access	documents FK references folders. signatures FK references documents and users. access controls Many-to-Many.
reports	work_reports, report_tasks, notifications, audit_logs	work_reports FK references users. notifications polymorphic ref. audit_logs append-only.

8.2 Core Table Schemas

users table

Column	Type	Constraint	Notes
id	UUID	PK, DEFAULT gen_random_uuid()	Primary key
full_name	VARCHAR(150)	NOT NULL	Employee full name
email	VARCHAR(200)	NOT NULL, UNIQUE	Login email, used for notifications
phone	VARCHAR(20)	NULLABLE	For SMS notifications
department	VARCHAR(100)	NOT NULL	HR, Finance, Operations, etc.
designation	VARCHAR(100)	NULLABLE	Job title
azure_ad_id	VARCHAR(200)	UNIQUE	Maps to Azure AD B2C object ID
is_active	BOOLEAN	DEFAULT true	Soft disable without deleting
avatar_url	TEXT	NULLABLE	Azure Blob URL of profile photo
created_at	TIMESTAMPTZ	DEFAULT NOW()	

updated_at	TIMESTAMPTZ	AUTO-UPDATE	Trigger-maintained
deleted_at	TIMESTAMPTZ	NULLABLE	Soft delete -- NULL means active

tasks table

Column	Type	Constraint	Notes
id	UUID	PK	
title	VARCHAR(200)	NOT NULL	Min 5 chars (app-level)
description	TEXT	NULLABLE	Max 2000 chars
created_by	UUID	FK references users	Auto from session
priority	task_priority	NOT NULL	ENUM: critical high normal low
status	task_status	NOT NULL	ENUM: new assigned in_progress pending_review approved rejected
due_date	TIMESTAMPTZ	NOT NULL	Must be future (app-level)
completed_at	TIMESTAMPTZ	NULLABLE	Set on status change to pending_review
delay_minutes	INT	GENERATED ALWAYS AS (EXTRACT(EPOCH FROM (completed_at - due_date))/60)	Computed column
efficiency_score	DECIMAL(5,2)	NULLABLE	Stored on approval
approved_by	UUID	FK references users, NULLABLE	
approved_at	TIMESTAMPTZ	NULLABLE	
rejection_reason	TEXT	NULLABLE	Required if rejected (app-level)
created_at	TIMESTAMPTZ	DEFAULT NOW()	
deleted_at	TIMESTAMPTZ	NULLABLE	Soft delete

9. API Specification

All endpoints use REST conventions built with Python FastAPI. Base URL: <https://api.fieldops.yourdomain.com/v1>. Authentication: Bearer JWT in Authorization header. FastAPI auto-generates interactive docs at /docs (Swagger UI) and /redoc. All request/response schemas are enforced by Pydantic v2 models.

Standard Error Response: { error: { code: 'VALIDATION_ERROR', message: 'due_date must be a future date', field: 'due_date' } }

9.1 Authentication

Method	Endpoint	Auth	Description
POST	/auth/login	None	Email + password login. Returns access_token (15min) + refresh_token (7d).
POST	/auth/refresh	Refresh token	Exchange refresh token for new access token.
POST	/auth/logout	Bearer	Invalidates current session/refresh token.
GET	/auth/me	Bearer	Returns current user profile and roles.

9.2 Tasks API

Method	Endpoint	Description + Key Params
GET	/tasks	List tasks. Query: ?status=&priority=&assigned_to=&page=&limit=&sort=due_date:asc
POST	/tasks	Create task. Body: {title, description, assigned_to[], priority, due_date, tags[]}
GET	/tasks/:id	Get single task with assignees, comments, attachments, approval history.
PATCH	/tasks/:id	Update task fields. Only creator, assigned supervisor, or admin.
DELETE	/tasks/:id	Soft delete. Admin only.
PATCH	/tasks/:id/status	Update status. Body: {status, note}. Triggers workflow + notifications.
POST	/tasks/:id/approve	Approve task. Body: {action: approved rejected, signature_data, rejection_reason?}
POST	/tasks/:id/comments	Add comment. Body: {text, attachments[]}
GET	/tasks/reports/efficiency	Efficiency report. Query: ?user_id=&from=&to=&department=

9.3 Service Calls API

Method	Endpoint	Description
--------	----------	-------------

GET	/service-calls	List. Filters: ?status=&priority=&assigned_to=&coordinator_id=&sla_breached=true
POST	/service-calls	Create. Body: {client_name, client_contact, issue_type, priority, description, sla_hours}
GET	/service-calls/:id	Full detail with history timeline.
PATCH	/service-calls/:id/assign	Coordinator assigns. Body: {technician_id, notes}. SMS sent to technician.
PATCH	/service-calls/:id/status	Update status. Body: {status, notes, attachments[]}
POST	/service-calls/:id/close	Coordinator closes. Body: {resolution_summary, signature_data}. Auto-email to client.
GET	/service-calls/reports/sla	SLA compliance report. Query: ?from=&to=&priority=

10. Security Architecture

Security Domain	Implementation
Authentication	Azure AD B2C with JWT. Access tokens expire in 15 minutes. Refresh tokens in 7 days. MFA supported and enforceable per role.
Authorisation	Role claims embedded in JWT. Every API endpoint validates role and ownership. Middleware-enforced -- no business logic bypasses.
Data in Transit	TLS 1.3 enforced on all endpoints. HSTS headers. Certificate managed by Azure.
Data at Rest	Azure SQL Transparent Data Encryption (TDE) enabled. Blob Storage server-side encryption (SSE) with Microsoft-managed keys.
Input Validation	All API inputs validated with Zod schema before processing. Parameterised queries only -- no string concatenation. HTML inputs sanitised.
File Upload Security	File type whitelist (PDF, DOCX, XLSX, PNG, JPG). Max 50MB. Antivirus scan via Azure Defender for Storage. Files stored with random UUID names.
Audit Logging	Every write operation logged to append-only audit_logs table: actor, action, before/after state, IP, timestamp. Logs cannot be deleted by any role.
Rate Limiting	100 requests/minute per authenticated user. 10 requests/minute for auth endpoints (brute-force protection). 429 responses with Retry-After header.
CORS	Strict origin whitelist. Credentials mode enabled only for known frontend domains.
Secrets Management	All secrets (DB credentials, SendGrid API keys, Twilio tokens) stored in Azure Key Vault. Never in code or environment variables directly.

11. Project Timeline & Sprint Plan

11.1 Phase Overview

Phase	Duration	Deliverables
Phase 1 - Foundation	Weeks 1-4	Azure setup, CI/CD pipeline, Auth (Azure AD B2C + python-jose), FastAPI project scaffold, Alembic DB migrations v1, core React UI shell, role-based navigation, user management module.
Phase 2 - Core Operations	Weeks 5-10	Task Manager (full), Expenses Manager (full), Service Manager (full). End-to-end workflows for all three. Unit and integration tests.
Phase 3 - Content & Reporting	Weeks 11-16	Documents Management, Work Reporting, Daily Task Advanced View, Graphical dashboards, efficiency calculator, export features.
Phase 4 - Automation & AI	Weeks 17-21	E-Sign Approvals, Chatbot integration (Azure Bot + OpenAI), Email/SMS automation (SendGrid + Twilio), notification engine.
Phase 5 - QA & Hardening	Weeks 22-24	Full UAT with stakeholders. Performance testing (target: under 500ms p95 API response). Security penetration test. Bug fixes.
Phase 6 - Launch	Weeks 25-26	Production deployment. User training sessions. Go-live monitoring. Handover documentation and runbooks.

11.2 Team Composition

Role	Count	Responsibilities
Project Manager	1	Sprint planning, stakeholder communication, risk tracking, timeline management
Full Stack Developer (Lead)	1	Architecture decisions, API design, code reviews, complex features
Frontend Developer	2	React UI, component library, responsive design, charting, e-sign canvas
Backend Developer	1	API implementation, database queries, Azure Functions, security
QA Engineer	1	Test case writing, manual and automated testing, UAT coordination
DevOps / Cloud Engineer	1	Azure setup, CI/CD, monitoring, deployment, infrastructure as code

12. Estimated Infrastructure Cost

Monthly cost estimates for a 50-user organisation on Microsoft Azure. Costs scale with usage and user count.

Service	Plan / Tier	Est. Monthly (USD)	Est. Monthly (INR)
Azure App Service	B2 (2 core, 3.5GB RAM)	~\$70	~Rs.5,800
Azure SQL Database	GP_Gen5_2 (2 vCores)	~\$185	~Rs.15,400
Azure Blob Storage	LRS, ~100GB + CDN	~\$5	~Rs.420
Azure Functions	Consumption (low volume)	~\$5	~Rs.420
Azure AD B2C	50 MAU (free tier)	\$0	Rs.0
Azure Bot Service	Standard (1000 msgs/day)	~\$15	~Rs.1,250
Azure App Insights	5GB/month included	~\$0-10	~Rs.0-830
SendGrid	Essentials (40k emails/month)	~\$20	~Rs.1,660
Twilio SMS	~500 SMS/month	~\$15	~Rs.1,250
Azure Cache for Redis	Basic C1	~\$25	~Rs.2,080
SSL Certificate	Azure managed	\$0	Rs.0
Total (Estimated)		~\$355/month	~Rs.29,500/month

Costs are estimates as of 2025 and vary by region. Azure free credits may offset initial months.

13. Risk Register

Risk	Prob.	Impact	Mitigation Strategy
Scope creep from stakeholders	High	High	Formal change control process. Signed module spec before development begins. Change requests logged and estimated separately.
E-sign legal validity in India	Medium	High	Implement under IT Act 2000 / ITA 2008. Consult legal team. Store OTP-verified signatures with full audit trail.
Azure cost overrun	Medium	Medium	Set billing alerts at 80% of budget. Use reserved instances after 3 months. Review monthly with DevOps.
User adoption resistance	High	High	Involve real end-users in UAT. Conduct training sessions. In-app onboarding tooltips. Dedicated support channel.
Chatbot wrong or hallucinated answers	Medium	Medium	Ground model on company documents only. Confidence threshold below 75% escalates to human. No financial data via chatbot.
Data migration from legacy systems	Medium	Medium	Audit all existing data sources in Week 1. Build ETL scripts with validation. Run parallel systems for 2 weeks post-launch.
Key developer unavailability	Low	High	Document all decisions in Confluence. Code review by 2 developers. No single point of knowledge failure.
SMS/Email delivery failure	Low	Medium	Delivery receipts via webhooks. Retry queue (3 attempts + exponential backoff). Fallback to in-app notification.
Security breach / data leak	Low	Critical	Penetration test before launch. Azure Defender enabled. No PII in logs. Regular security training for developers.

14. Testing Strategy

Test Type	Tools	Scope & Targets
Unit Tests	pytest + React Testing Library	All service layer functions, utility functions, Pydantic schemas, and React components in isolation. Target: above 80% code coverage.
Integration Tests	pytest + httpx (async test client)	All API endpoints: happy path and error cases. FastAPI TestClient. DB query accuracy with test DB fixtures.
End-to-End Tests	Playwright	Critical user journeys: Login to Create Task to Approve. Submit Expense to Approve. Create Service Call to Close.
Performance Tests	k6 + Azure Load Testing	500 concurrent users. Target: p95 API response under 500ms. Identify DB bottlenecks.
Security Tests	OWASP ZAP + Manual	OWASP Top 10. SQL injection, XSS, IDOR, auth bypass. Run before UAT.
UAT	Manual + Stakeholders	Real users run all module workflows. Acceptance criteria signed off per module.
Regression Tests	Playwright (automated)	Run on every merge to main branch via GitHub Actions CI pipeline.

15. Recommended Next Steps

1. Stakeholder sign-off on this document -- module priorities, phased rollout, and budget approval
2. UI/UX wireframes for all 8 modules -- recommend Figma with a shared component library
3. Azure subscription setup -- resource groups, Key Vault, CI/CD pipeline via GitHub Actions
4. Database schema finalisation -- review with dev team, create Alembic migration scripts
5. Identify real end-users for each role -- interview and document edge cases
6. Vendor account setup -- SendGrid, Twilio, OpenAI/Azure OpenAI -- test connectivity
7. Sprint 1 kickoff -- Auth + DB + Core UI shell (Weeks 1-2)

This document is prepared for demonstration and internal planning purposes. All cost figures are estimates. -- Enterprise Field Operations Platform Team
